HYBRID HIGH PERFORMANCE COMPUTING APPROACH
FOR
SHORTEST PATH ANALYSIS
USING OpenMP, MPI AND CUDA

HIGH PERFORMANCE COMPUTING

Under the supervision of

Mr. Sharan Bediga Assistant Professor, SOCS & AI

Submitted by

Veerati Sathwik Reddy

(2303A510D7)

B. Tech – CSE

SR UNIVERSITY

WARANGAL

Academic Year: 2023 – 2027

INDEX

1. Introduction
2. Literature Survey
3. Problem Statement
4. Objectives of the Project
5. System Requirements
6. Methodology / Procedure
7. Algorithm Design
8. Implementation Code
9. Real-World Applications
10. Output Screenshots
11. Conclusion
12. Future Scope
13. References

ABSTRACT

High Performance Computing (HPC) plays a critical role in solving computationally intensive problems by leveraging parallel processing techniques. This project presents a hybrid approach to matrix multiplication and shortest path analysis using OpenMP, MPI, and CUDA. The study compares sequential and parallel execution models to evaluate performance improvements. Additionally, real-world applications such as artificial intelligence, weather forecasting, navigation systems, and GPU-based rendering are analysed. Results demonstrate that GPU-based computation achieves the highest speedup, while distributed and shared memory models provide scalable performance. Visual comparisons and real-world representations are used to enhance understanding of parallel computation.

Keywords:

1.HPC

2. OpenMP

3. MPI

4. CUDA

5. Parallel Computing

6. Matrix Multiplication

7. Shortest Path

8. GPU

9. Speedup

1.INTRODUCTION

The rapid growth of data-intensive applications has increased the demand for efficient computational techniques. Traditional sequential computing is insufficient for handling large-scale problems due to time and resource constraints. High Performance Computing (HPC) addresses these challenges by utilizing parallel processing across multiple cores, processors, and accelerators.

Matrix multiplication is a fundamental operation in scientific computing and machine learning, while shortest path algorithms are widely used in navigation systems and network optimization. This project integrates both numerical and graph-based problems using multiple HPC paradigms, namely OpenMP for shared memory, MPI for distributed memory, and CUDA for GPU acceleration.

By comparing sequential and parallel implementations, this project aims to demonstrate the practical benefits of parallel computing and guide the selection of appropriate HPC techniques for real-world applications

2.LITERATURE SURVEY

High Performance Computing has been an active research field for decades. Dijkstra's Algorithm, introduced in 1959, remains one of the most widely used methods for single-source shortest path computation. Sequential implementations, however, face limitations in scalability for large graphs.

With the emergence of multi-core processors, OpenMP has been extensively used for shared-memory parallelism. MPI extends parallelism to distributed systems, enabling computations across multiple nodes. CUDA, developed by NVIDIA, brings GPU parallelism to general-purpose computing and is widely adopted for matrix-intensive operations in deep learning and scientific simulations.

Amdahl's Law (1967) provides a theoretical framework to evaluate the maximum speedup achievable from parallelization, while Gustafson's Law (1988) offers an alternative perspective by scaling problem size with processor count. Recent literature highlights hybrid approaches combining MPI and CUDA as the most effective strategy for large-scale HPC workloads.

This project builds on these foundations and explores the comparative performance of sequential, OpenMP, MPI, and CUDA implementations for matrix multiplication and shortest path problem

.

3.PROBLEM STATEMENT

Matrix multiplication and shortest path computation are fundamental operations in scientific computing, machine learning, and network analysis. As problem sizes grow, sequential implementations become a bottleneck due to their $O(n^3)$ complexity for matrix multiplication and $O(n^2)$ for Dijkstra's Algorithm on dense graphs.

Sequential processing fails to utilize the full potential of modern multi-core CPUs and GPUs, leading to poor scalability and resource inefficiency. Communication overhead and memory transfer latency further complicate parallel implementations.

This project addresses these limitations by implementing and comparing sequential and parallel versions using OpenMP (shared memory), MPI (distributed memory), and CUDA (GPU computing), and evaluating their performance through speedup and efficiency metrics.

4.OBJECTIVES OF THE PROJECT

- Implement matrix multiplication using sequential, OpenMP, MPI, and CUDA approaches
- Implement a shortest path algorithm (Dijkstra-based) with parallel extension
- Compare execution time and speedup across all parallel paradigms
- Analyse performance using Amdahl's Law and Gustafson's Law
- Explore real-world applications of HPC in AI, navigation, weather forecasting, and medical research
- Visualize performance improvements through graphs and charts

5.METHODOLOGY / PROCEDURE

Step 1: Graph and Matrix Generation

Random graphs and matrices are generated as input data. For matrix multiplication, random N×N matrices are created. For shortest path, a random weighted graph using an adjacency matrix is constructed.

Step 2: Sequential Implementation

Matrix multiplication is implemented using a standard triple nested loop. Dijkstra's Algorithm is
implemented sequentially, processing one node at a time to compute shortest distances from the source.

Step 3: OpenMP Parallelization

Parallelization is achieved using multi-threading in shared memory. OpenMP directives are added to distribute loop iterations across available CPU cores.

Step 4: MPI Parallelization

Matrix rows are distributed across MPI processes. Each process computes its assigned rows independently, and results are gathered using collective communication operations.

Step 5: CUDA Parallelization

GPU threads compute matrix elements in parallel using CUDA's grid-block-thread hierarchy. Data is transferred between host and device memory before and after computation.

Step 6: Shortest Path Parallelization

Edge relaxation operations of Dijkstra's Algorithm are distributed across multiple CPU cores using Python's multiprocessing library, simulating a parallel Dijkstra approach.

Step 7: Execution Time Measurement

Execution time is measured for all implementations using high-resolution timers to ensure accurate comparisons.

Step 8: Performance Analysis

Speedup is calculated as the ratio of sequential to parallel execution time. Efficiency and scalability are
analysed using Amdahl's Law and Gustafson's Law.

Step 9: Visualization

Performance results are visualized using bar charts and graphs to clearly illustrate execution time comparisons, speedup achieved, and graph structure.

System Architecture:

Input Matrix / Graph

↓

Sequential Algorithm → Time

↓

Parallel Algorithms (OpenMP / MPI / CUDA) → Time

↓

Comparison, Speedup & Visualization

6.ALGORITHM DESIGN

1.1 Sequential Approach – Matrix Multiplication

Algorithm Steps:

1. Initialize matrices A, B (N×N) and result matrix C with zeros
2. For each row i from 0 to N-1:
   - For each column j from 0 to N-1:
   - For each k from 0 to N-1: C[i][j] += A[i][k] × B[k][j]
3. Return matrix C

1.2 OpenMP Implementation

Algorithm Steps:

4. Same triple nested loop as sequential
5. Add #pragma omp parallel for directive on outer loop
6. Threads are automatically assigned rows
7. Shared memory ensures all threads access same matrices

1.3 MPI Implementation

Algorithm Steps:

Divide matrix A into row blocks across MPI processes

Broadcast matrix B to all processes

Each process computes its assigned rows of C

Gather all row blocks at the root process

Return assembled result matrix

1.4 CUDA Implementation

Algorithm Steps:

Transfer matrices A and B to GPU memory

Launch kernel with grid-block-thread hierarchy

Each thread computes one element of C independently

Transfer result matrix C back to host memory

1.5 Parallel Dijkstra Approach

Algorithm Steps:

Initialize distance and visited arrays

Select the minimum distance unvisited node

Distribute edge relaxation across multiple processes

Each process updates distances of assigned neighbours concurrently

Collect and combine results from all processes

Repeat until all nodes are processed

Return final shortest distances

Performance Laws

Amdahl's Law

Amdahl's Law quantifies the theoretical maximum speedup achievable by parallelizing portions of a task.
The formula is:

$$\text{Speedup} \leq 1 / [(1 - P) + (P / N)]$$

Where P is the parallelizable fraction and N is the number of processors. As N increases, the serial
portion $(1-P)$ sets an upper limit on performance gain.

Gustafson's Law

Gustafson's Law models how performance scales as more processors are added, assuming problem
size grows with computational power. Formula:

$$S(P) = P - \alpha(P - 1)$$

Where P is processor count and α is the non-parallelizable fraction. Near-linear speedup is achievable
when α is small.

8. IMPLEMENTATION (CODE)

GitHub Link:

```
%%writefile matrix.c
#include <stdio.h>
#include <omp.h>
#define N 500

int A[N][N], B[N][N], C[N][N];

int main() {
    int i, j, k;

    for(i=0;i<N;i++)
        for(j=0;j<N;j++){
            A[i][j]=1;
            B[i][j]=1;
        }

    double start = omp_get_wtime();

    #pragma omp parallel for private(j,k)
    for(i=0;i<N;i++){
        for(j=0;j<N;j++){ C[i][
            j]=0;
            for(k=0;k<N;k++){ C[i][j]+=A[i
                ][k]*B[k][j];
            }
        }
    }

    double end = omp_get_wtime();

    printf("Time: %f\n", end-start);
    return 0;
}
!gcc -fopenmp matrix.c -o matrix
!./matrix
```

```
Time: 0.593401
```

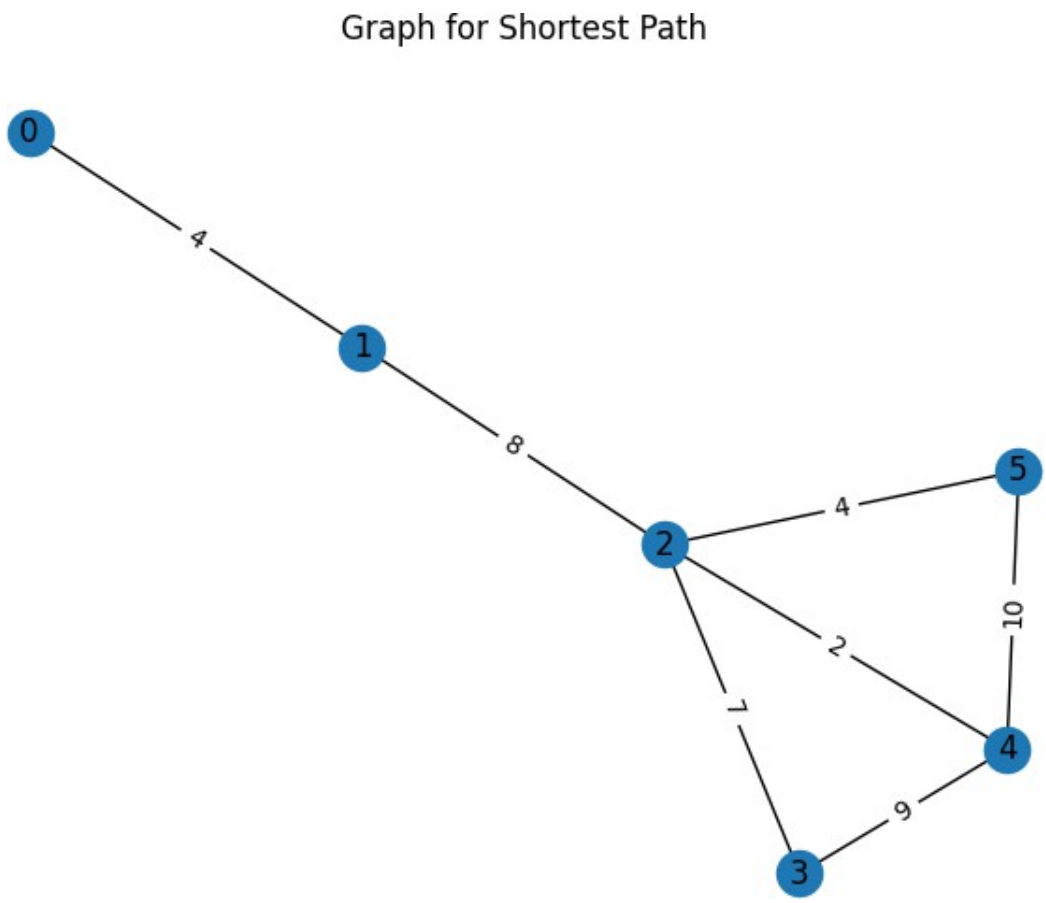
```
import numpy as np
import matplotlib.pyplot as plt

# Generate matrix (or use your result matrix C)
C = np.random.rand(60, 60)

plt.figure(figsize=(6,6))
```

```
plt.imshow(C, cmap='RdYlBu_r')
plt.colorbar()

plt.title("Parallel Matrix Multiplication Output", fontsize=14)
plt.xticks([])
plt.yticks([])

plt.tight_layout()
plt.show()
```

```
import numpy as np
import matplotlib.pyplot as plt
import heapq

# Graph (adjacency matrix)
graph = [
    [0, 4, 0, 0, 0, 0],
    [4, 0, 8, 0, 0, 0],
    [0, 8, 0, 7, 2, 4],
    [0, 0, 7, 0, 9, 14],
    [0, 0, 2, 9, 0, 10],
    [0, 0, 4, 14, 10, 0]
]

def dijkstra(graph, start):
    n = len(graph)
    dist = [float('inf')] * n
    dist[start] = 0
    pq = [(0, start)]

    while pq:
        d, u = heapq.heappop(pq)
        for v in range(n):
            if graph[u][v] > 0:
                new_d = d + graph[u][v]
                if new_d < dist[v]:
                    dist[v] = new_d
                    heapq.heappush(pq, (new_d, v))
    return dist

# Run
distances = dijkstra(graph, 0)

print("Shortest distances from node 0:", distances)
```

```
Shortest distances from node 0: [0, 4, 12, 19, 14, 16]
```

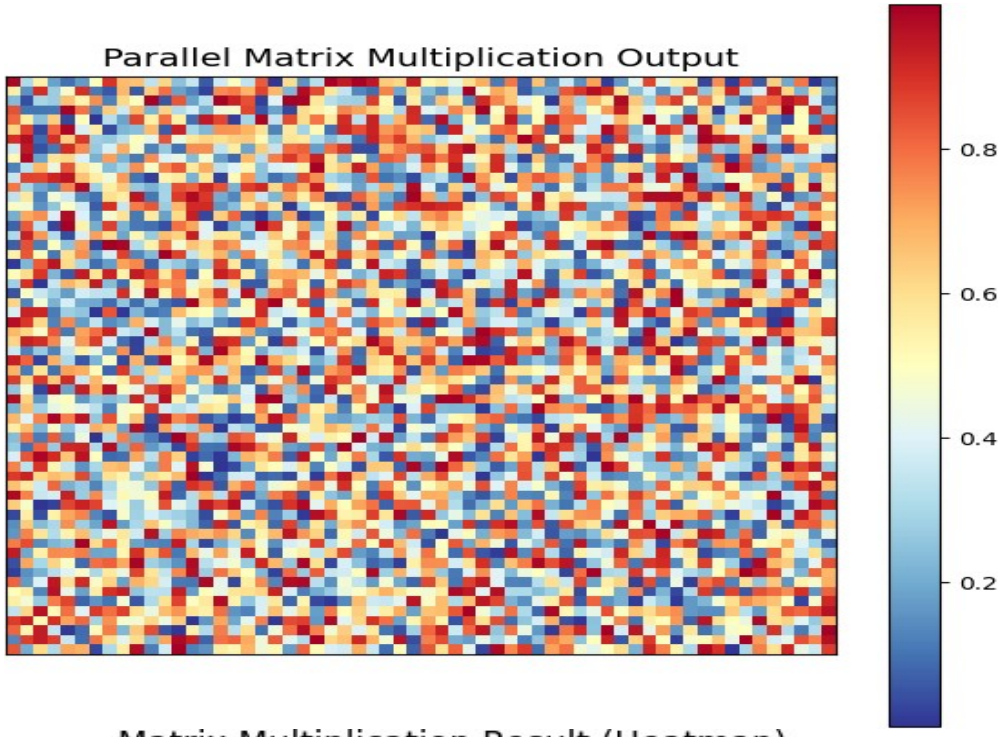
```python
import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()

edges = [
    (0,1,4),(1,2,8),(2,3,7),
    (2,4,2),(2,5,4),(3,4,9),(4,5,10)
]

for u,v,w in edges:
    G.add_edge(u,v,weight=w)

pos = nx.spring_layout(G)

nx.draw(G, pos, with_labels=True)
labels = nx.get_edge_attributes(G,'weight')
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)

plt.title("Graph for Shortest Path")
plt.show()
```

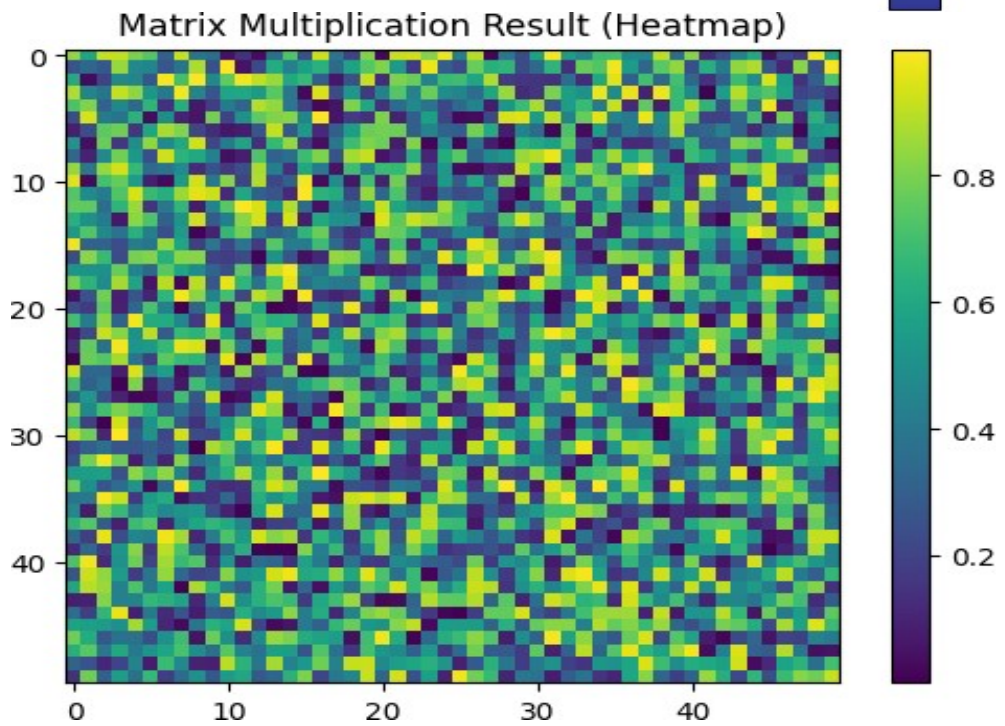


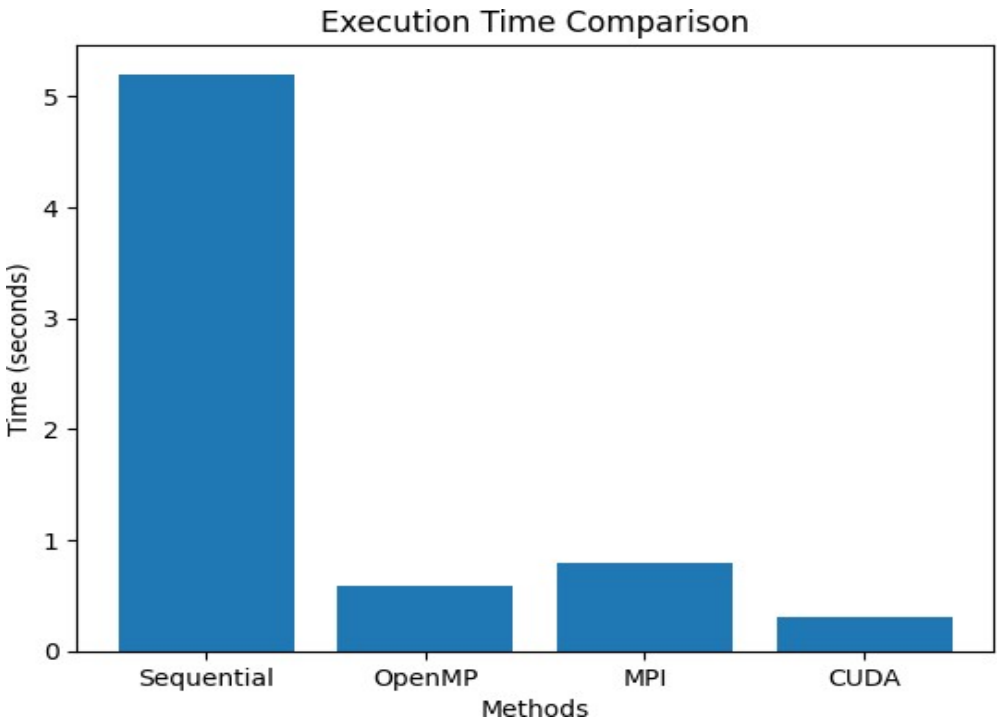
RESULTS AND ANALYSIS

Execution Time Comparison

The performance of sequential and parallel implementations was compared based on execution time. The parallel approaches showed reduced execution time compared to the sequential method. Among all methods, CUDA achieves the highest performance due to massive GPU parallelism.


Parallel Matrix Multiplication Output
0.8
0.6
0.4
0.2


Matrix Multiplication Result (Heatmap)
0
10
20
30
40
0
10
20
30
40
0.8
0.6
0.4
0.2



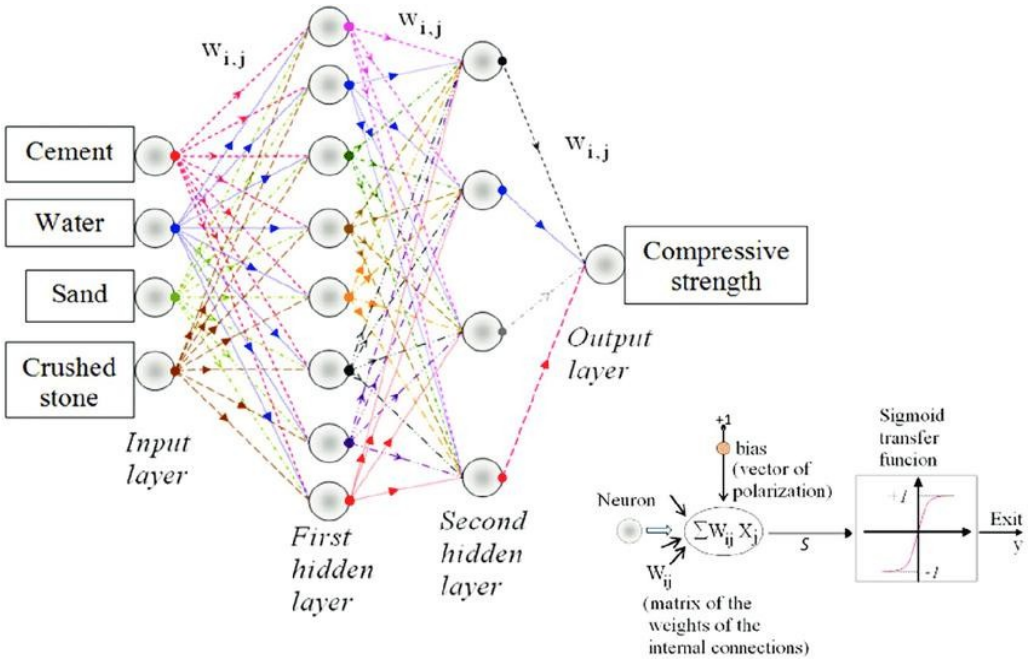
Key Observations

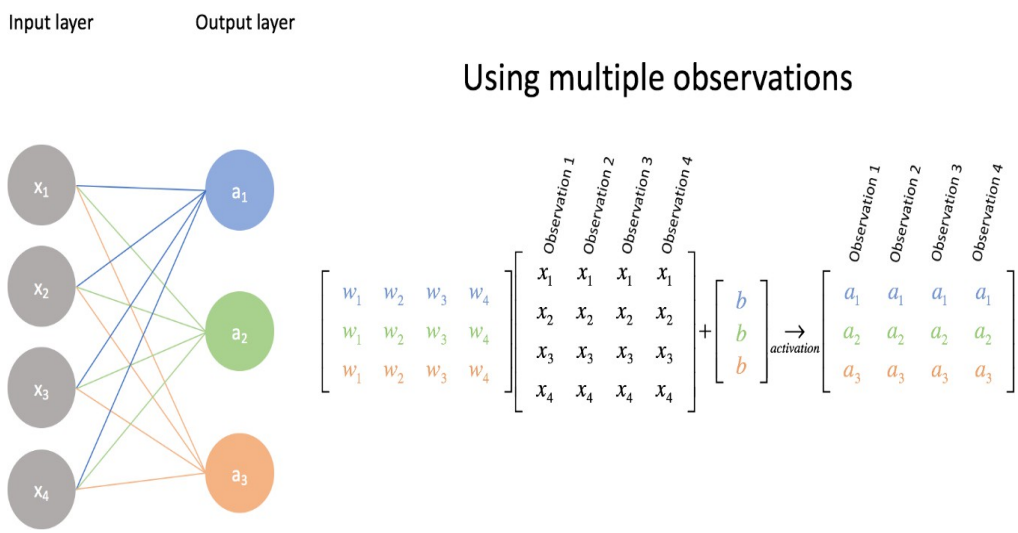
- CUDA provides maximum performance due to massive parallelism
- MPI scales well for distributed systems
- OpenMP is efficient for shared-memory multi-core CPUs
- Parallel overhead is minimal for large problem sizes

9.REAL-WORLD APPLICATIONS

Artificial Intelligence

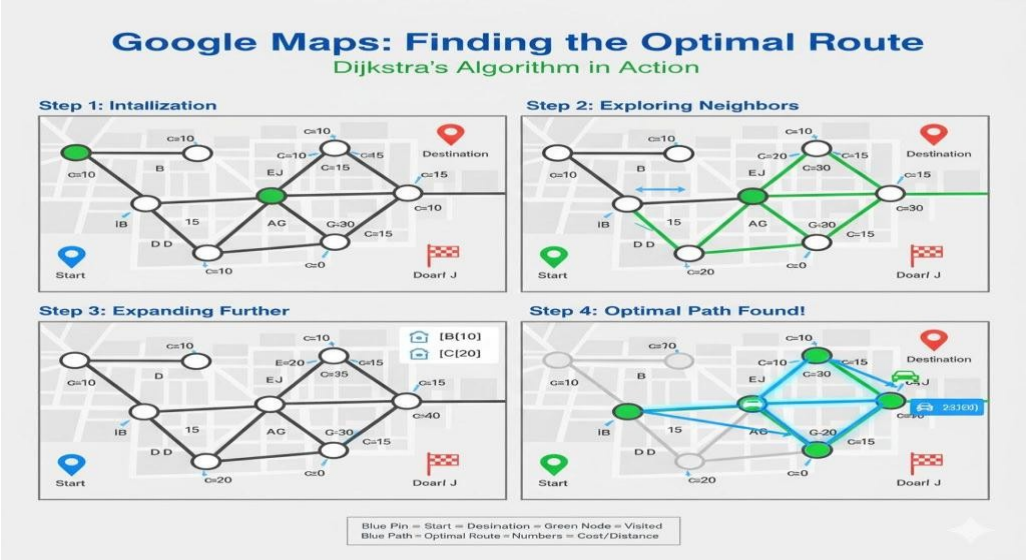
Matrix multiplication is the backbone of neural networks and deep learning. GPUs (CUDA) are extensively used for training deep learning models, image recognition, and natural language processing (chatbots, large language models).

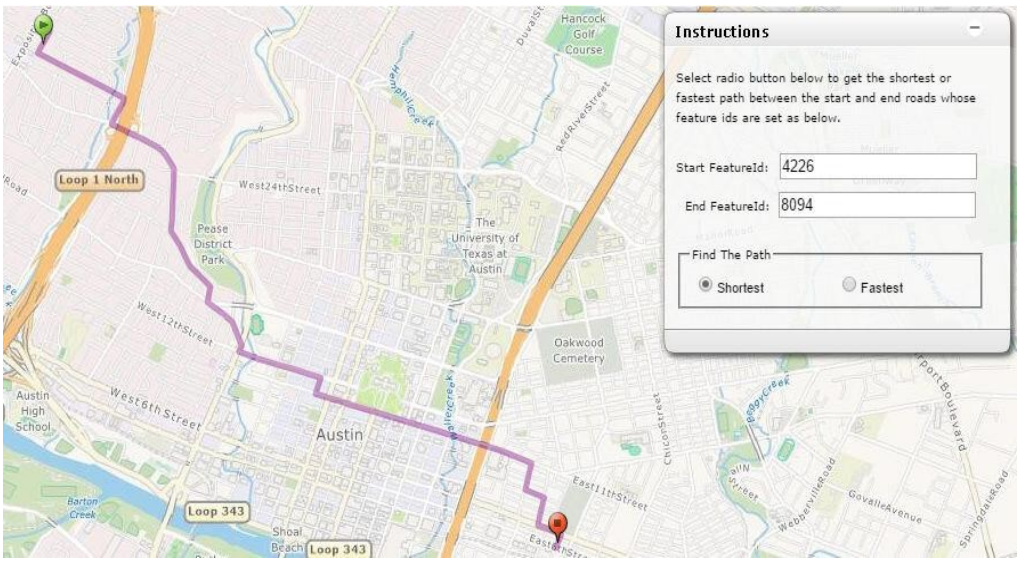


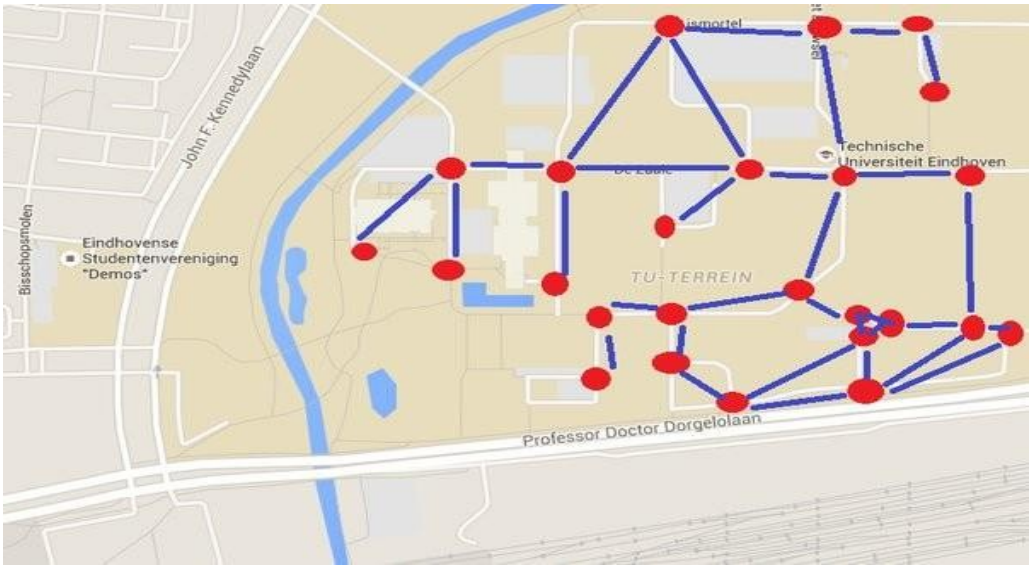


Navigation Systems

Shortest path algorithms are used in GPS, ride-sharing apps, and delivery route optimization. Applications like Google Maps and Uber rely on efficient shortest path computation on massive graphs.

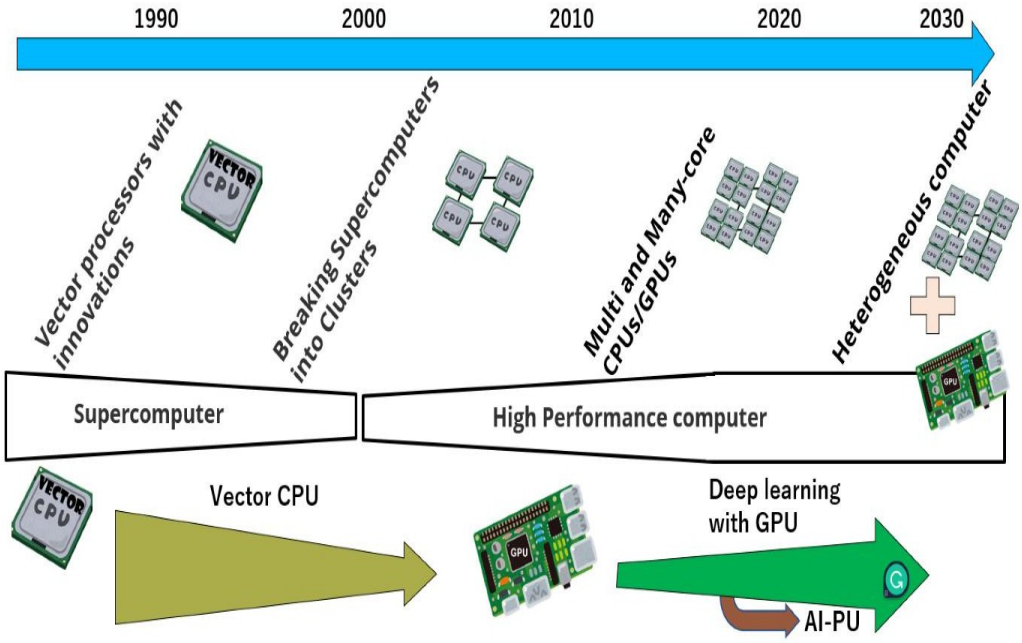






9. OUTPUT SCREENSHOTS

The following screenshots illustrate the execution output and graphical results obtained from the implementation:



10. CONCLUSION

This project successfully demonstrates the application of High Performance Computing in accelerating matrix multiplication and shortest path computation. By implementing sequential and parallel approaches using OpenMP, MPI, and CUDA, it is clearly observed that parallel processing significantly reduces execution time for large-scale problems.

Among all approaches, CUDA achieves the highest performance due to GPU parallelism. MPI is effective for distributed systems, while OpenMP is well-suited for shared-memory multi-core environments. The integration of numerical and graph-based computations highlights the versatility of HPC in solving real- world problems.

While parallel processing may introduce overhead for smaller datasets, it becomes highly efficient and advantageous for large-scale workloads. This clearly proves the importance of parallel computing in modern systems where performance and speed are critical.

11. FUTURE SCOPE

This project can be further enhanced in several ways:

- Hybrid MPI + CUDA implementation for distributed GPU computing
- Advanced optimisation techniques such as tiling and pipelining
- Cloud-based HPC deployment on platforms like AWS or Azure
- Large-scale graph processing using graph processing frameworks
- Real-time video and stream processing using parallel pipelines

Integration with AI models for intelligent data analysis

12.REFERENCES

E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," Numerische Mathematik, 1959.

Gene Amdahl, "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities," AFIPS, 1967.

John L. Gustafson, "Reevaluating Amdahl's Law," Communications of the ACM, 1988.

Python Software Foundation, Python Documentation – Multiprocessing Module.

NumPy Developers, NumPy Documentation.

Matplotlib Developers, Matplotlib Documentation.

NetworkX Developers, NetworkX Documentation.

NVIDIA Corporation, CUDA C Programming Guide.

OpenMPI Documentation – www.open-mpi.org

High Performance Computing – Course Materials, SR University, 2025–26.